

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: CSA1407

COURSE NAME: COMPILER DESIGN

COURSE OUTCOME COVERED

CO5: Develop code optimization and Code Generation using DAG representation with data flow analysis to produce optimized target code. (BL3)

Assessment Tool Weight-age: 40%

QUESTIONS: 5

Total Marks:50

Annexure A

SIMULATION TASK

Code of Conduct:

I JEROLD SAMUEL J (192571052) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: JEROLD SAMUEL J

Criteria	Weight age (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Conceptual Understanding	20%	Demonstrates strong understanding of underlying concepts in the simulation	Shows good understanding with minor gaps	Partial understanding; some misconceptions	Lacks understanding of key concepts

Model Design	25%	Develops accurate, well-structured simulation/model independently	Develops correct model with minor errors	Model is incomplete or requires guidance	Unable to develop a proper model
Tool Usage	20%	Uses simulation tools effectively with correct setup and execution	Uses tools with minor errors or guidance	Limited ability to use tools properly	Unable to use simulation tools
Analysis of Results	20%	Provides clear, logical analysis and meaningful interpretation of results	Analysis is reasonable with minor gaps	Superficial analysis; limited interpretation	Unable to analyze or interpret results
Documentation	15%	Presents results clearly with proper documentation and structured reporting	Documentation is adequate with minor issues	Poorly organized or incomplete documentation	No proper documentation or presentation

Annexure A

SIMULATION TASK

QUESTIONS

1. Given the three-address code sequence:

```
t1 = x * 1
t2 = t1 + y
t3 = t2 - 0
t4 = t3 * 0
if t4 != 0 goto L1
t5 = y + y
```

Simulate the **peephole optimization** process on this sequence. Produce the optimized instruction sequence and justify each optimization step applied, including algebraic simplification, constant folding, and dead code elimination.

2. Partition the following intermediate code into **basic blocks** and construct the corresponding **control flow graph**. Identify all leader statements and mention the rules used to identify them:

```
(1) x = a + b
(2) y = x * c
(3) if y > 50 goto 6
(4) z = y - d
(5) goto 7
(6) z = y + d
(7) w = z * 2
```

3. Construct the **DAG** for the following basic block:

```
p = a + b
q = a + b
r = p * q
s = a + b
t = r + s
```

Using the DAG, identify all **common sub-expressions**. Then simulate a simple code generator to produce optimized target code and explain the register allocation decisions at each step.

4. For a target machine with two registers **R0** and **R1**, simulate the working of a **simple code generator** for the code:

```
t1 = a + b
t2 = c * d
t3 = t1 - t2
t4 = e + f
t5 = t3 / t4
```

Show the **register descriptor** and **address descriptor** after each instruction is processed.

5. For a **RISC-style target machine** with instructions **LOAD**, **STORE**, **ADD**, **SUB**, **MUL**, **MOV**, simulate target code generation for the expression:

$(a - b) * (c + d) - e$

Show the complete sequence of target instructions generated. Also explain the **runtime storage management** and **stack frame layout** used during the computation.

Given the three-address code sequence:

```
t1 = a + b,  
t2 = t1 + 0,  
t3 = t2 * 1,  
if t3 == 0 goto L1,  
t4 = t3 + t3
```

ANSWERS:

1. Peephole Optimization

Given:

```
t1 = x * 1  
t2 = t1 + y  
t3 = t2 - 0  
t4 = t3 * 0  
if t4 != 0 goto L1  
t5 = y + y
```

Step 1: Algebraic simplification

- $x * 1 \rightarrow x$
- $t2 - 0 \rightarrow t2$

So:

```
t1 = x  
t2 = t1 + y  
t3 = t2  
t4 = t3 * 0  
if t4 != 0 goto L1  
t5 = y + y
```

Step 2: Constant folding

anything * 0 = 0

Therefore:

t4 = 0

Then:

if t4 != 0 goto L1

becomes:

if 0 != 0 goto L1

This condition is always false, so the jump can be removed.

Step 3: Copy propagation

Since:

t1 = x

t3 = t2

we can simplify:

t2 = x + y

Step 4: Dead code elimination

Since the conditional jump is never taken, L1 is unreachable from this sequence. Also, t5 is needed only if its result is used later; assuming it is not used, it can be eliminated.

Optimized code

t2 = x + y

Optimizations used: algebraic simplification → constant folding → copy propagation → dead code elimination.

2. Basic Blocks and Control Flow Graph

Given:

- (1) $x = a + b$
- (2) $y = x * c$
- (3) if $y > 50$ goto 6
- (4) $z = y - d$
- (5) goto 7
- (6) $z = y + d$
- (7) $w = z * 2$

Leaders

Rules for finding leaders:

1. The first instruction is a leader.
2. The target of a goto/conditional goto is a leader.
3. The instruction immediately after a goto/conditional goto is a leader.

Therefore:

- 1 $\rightarrow$ first instruction
- 4 $\rightarrow$ statement after conditional jump (3)
- 6 $\rightarrow$ target of conditional jump
- 7 $\rightarrow$ target of goto (5)

Basic Blocks

B1:

1. $x = a + b$
2. $y = x * c$
3. if $y > 50$ goto 6

B2:

4. $z = y - d$
5. goto 7

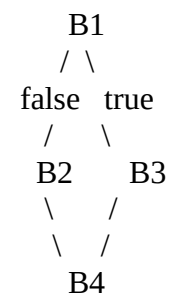
B3:

6. $z = y + d$

B4:

7. $w = z * 2$

Control Flow Graph



Edges:

- B1 $\rightarrow$ B2
- B1 $\rightarrow$ B3
- B2 $\rightarrow$ B4
- B3 $\rightarrow$ B4

3. DAG and Common Sub-expressions

Given:

- $p = a + b$
 $q = a + b$

$R0 = a + b$
 $R1 = R0 * R0$
 $R1 = R1 + R0$

Finally:

$t = R1$

The important idea is reuse the result $a+b$ instead of recalculating it.

4. Code Generation with Two Registers

Given:

$t1 = a + b$
 $t2 = c * d$
 $t3 = t1 - t2$
 $t4 = e + f$
 $t5 = t3 / t4$

Assume two registers: $R0$ and $R1$.

Step 1

$t1 = a + b$
 $R0 = a$
 $R0 = R0 + b$

Descriptors:

Register Descriptor:

$R0 \rightarrow t1$

$R1 \rightarrow \text{empty}$

Address Descriptor:

$t1 \rightarrow R0$
 $a \rightarrow \text{memory}$
 $b \rightarrow \text{memory}$

Step 2

$t2 = c * d$
 $R1 = c$
 $R1 = R1 * d$

Descriptors:

$R0 \rightarrow t1$

$R1 \rightarrow t2$

$t1 \rightarrow R0$

$t2 \rightarrow R1$

Step 3

$t3 = t1 - t2$
 $R0 = R0 - R1$

Now:

$R0 \rightarrow t3$

$R1 \rightarrow t2$

$t2$ is no longer needed, so $R1$ can be reused.

Step 4

$t4 = e + f$
 $R1 = e$
 $R1 = R1 + f$

Now:

$R0 \rightarrow t3$

$R1 \rightarrow t4$

Step 5

$t5 = t3 / t4$

$R0 = R0 / R1$

Final:

$R0 \rightarrow t5$

$R1 \rightarrow t4$

Optimized target instructions:

LOAD R0, a

ADD R0, b

LOAD R1, c

MUL R1, d

SUB R0, R1

LOAD R1, e

ADD R1, f

DIV R0, R1

5. RISC Target Code Generation

Expression:

$(a - b) * (c + d) - e$

We first divide it into parts:

$t1 = a - b$

$t2 = c + d$

$t3 = t1 * t2$

$t4 = t3 - e$

Target code

LOAD R0, a

LOAD R1, b

SUB R0, R1 ; $R0 = a - b$

LOAD R1, c

ADD R1, d ; $R1 = c + d$

MUL R0, R1 ; $R0 = (a-b)*(c+d)$

SUB R0, e ; $R0 = (a-b)*(c+d)-e$

STORE result, R0

Runtime storage management

During execution, the program needs memory for:

- Local variables
- Temporary values
- Function parameters
- Return address
- Saved registers

Simple stack frame

```
-----  
| Return address |  
-----  
| Saved registers |  
-----  
| Parameters      |  
-----  
| Local variables |  
-----  
| Temporaries     |  
-----
```

The stack frame is created when a function starts and removed when the function returns.

Final Three-Address Code

You also gave:

```
t1 = a + b  
t2 = t1 + 0  
t3 = t2 * 1  
if t3 == 0 goto L1  
t4 = t3 + t3
```

Optimization

1. Algebraic simplification

```
t2 = t1 + 0
```

becomes:

```
t2 = t1
```

And:

```
t3 = t2 * 1
```

becomes:

```
t3 = t2
```

2. Copy propagation

Therefore:

```
t3 = a + b
```

The condition becomes:

```
if a + b == 0 goto L1
```

3. Final optimized code

```
t1 = a + b
```

```
if t1 == 0 goto L1
```

```
t4 = t1 + t1
```

RUBRICS FOR CALCULATION

Simulation tools

Criteria	Weight age (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Conceptual Understanding	20%	Demonstrates strong understanding of underlying concepts in the simulation	Shows good understanding with minor gaps	Partial understanding; some misconceptions	Lacks understanding of key concepts
Model Design	25%	Develops accurate, well-structured simulation/model independently	Develops correct model with minor errors	Model is incomplete or requires guidance	Unable to develop a proper model
Tool Usage	20%	Uses simulation tools effectively with correct setup and execution	Uses tools with minor errors or guidance	Limited ability to use tools properly	Unable to use simulation tools
Analysis of Results	20%	Provides clear, logical analysis and meaningful interpretation of results	Analysis is reasonable with minor gaps	Superficial analysis; limited interpretation	Unable to analyze or interpret results
Documentation	15%	Presents results clearly with proper documentation and structured reporting	Documentation is adequate with minor issues	Poorly organized or incomplete documentation	No proper documentation or presentation